LLD Practice Platform

A working prototype of a Low-Level Design (LLD) practice platform that helps learners design systems (e.g., Parking Lot, Elevator, Vending Machine), submit solutions, and receive explainable feedback.

Quick Start

```
# Install dependencies
npm install

# Run tests
npm test

# Start the development server
npm start
```

Open `http://localhost:3000` in your browser to use the platform.

Project Structure

```
src/
├── domain/                # Core domain (entities, interfaces, enums)
│   ├── enums/index.js
│   ├── interfaces/        # Port/protocol interfaces (DI Principle)
│   │   ├── FeedbackProvider.js
│   │   ├── ProblemRepository.js
│   │   ├── AttemptRepository.js
│   │   └── LearnerRepository.js
│   └── models/            # Domain entities
│       ├── Attempt.js
│       ├── AttemptHistory.js
│       ├── Feedback.js
│       ├── Learner.js
│       ├── LldProblem.js
│       └── Solution.js
├── application/           # Use cases and application services
│   ├── services/
│   │   ├── PracticeService.js
│   │   └── FeedbackService.js
│   └── use-cases/         # Individual use case classes
```

```
│       ├── StartAttempt.js
│       ├── SaveSolutionDraft.js
│       ├── SubmitAttempt.js
│       ├── GenerateFeedback.js
│       ├── ListProblems.js
│       └── GetAttemptHistory.js
├── infrastructure/        # Concrete implementations
│   ├── repositories/      # In-memory repositories
│   ├── feedback/
│   │   ├── RuleBasedFeedbackProvider.js   # Deterministic checks
│   │   ├── LLMBasedFeedbackProvider.js    # LLM-based (simulated/real)
│   │   └── FeedbackProviderChain.js       # Chain of Responsibility
│   └── problems/problemCatalog.js         # Pre-seeded LLD problems
├── api/                   # REST API layer
│   ├── routes/index.js
│   └── middleware/errorHandler.js
├── app.js                 # Express application setup
├── container.js           # Composition root (DI)
└── index.js               # Server entry point
public/
├── index.html
├── css/style.css
└── js/app.js
tests/
├── domain/
├── application/
├── infrastructure/
└── integration/
```

API Endpoints

Method	Endpoint	Description
GET	`/api/health`	Health check
GET	`/api/problems`	List all problems (optional `?tag=`)
GET	`/api/problems/:id`	Get a specific problem
POST	`/api/attempts`	Start a new attempt
GET	`/api/attempts/:id`	Get an attempt by ID
PUT	`/api/attempts/:id/solution`	Save/update a solution draft
POST	`/api/attempts/:id/submit`	Submit for feedback
POST	`/api/attempts/:id/feedback`	Generate feedback
GET	`/api/attempts?learnerId=`	Get attempt history

Method	Endpoint	Description
GET	`/api/learner/default`	Get default learner

Practice Flow

1. **Choose Problem** — Browse the catalog of LLD problems (Parking Lot, Elevator, Vending Machine, Library, Tic-Tac-Toe, LRU Cache)
2. **Think / Design** — Write your solution (class diagrams, pseudocode, or code) and explain your design decisions
3. **Submit** — Submit your solution for evaluation
4. **Get Feedback** — Receive structured feedback across 7 criteria (Modularity, SOLID, Patterns, Responsibilities, Relationships, Trade-offs, Completeness)
5. **Review** — View detailed scores, comments, and suggestions for improvement
6. **Try Again** — Start a new attempt on the same or different problem; view history of all past attempts

Feedback Criteria

Criterion	Description
Modularity	How well the design is broken into cohesive, single-purpose classes
SOLID Principles	Application of SRP, OCP, LSP, ISP, DIP
Design Patterns	Appropriate use of patterns (Strategy, Factory, Observer, etc.)
Responsibilities	Clear separation of concerns per class
Relationships	Proper modeling of inheritance, composition, aggregation, association
Trade-offs	Explicit analysis of design alternatives and their implications
Completeness	Coverage of all problem requirements and constraints

Technologies

- **Runtime**: Node.js (>=20)
- **API**: Express.js
- **Frontend**: Vanilla HTML/CSS/JavaScript

- **Storage**: In-memory (swapable repository pattern for future persistence)
- **Testing**: Jest + Supertest
- **ID Generation**: Node.js built-in `crypto.randomUUID()`

Key Design Decisions

See DESIGN_NOTE.md for the full design rationale and AI_USAGE.md for AI-assisted decisions.

Limitations

- In-memory storage (data is lost on restart); easily swappable via the repository interface for a database implementation
- LLM feedback provider includes a simulation mode; real LLM evaluation requires an OpenAI API key (`OPENAI_API_KEY` env var)
- Single-threaded Node.js runtime; concurrent feedback generation runs sequentially

License

Internal assignment — CipherSchools Hiring Assignment SEP 2026